

TD/TP 5 — State, TimedState et Trajectory

Composition, héritage public et slicing

MAM5 — Programmation C++ Durée : 2 heures

Objectif. Partir d'une classe-valeur numérique, ajouter un instant par héritage public, réutiliser des algorithmes par référence de base, observer le slicing, puis construire une trajectoire par composition. Le squelette est dans `code/seance5/`.

Organisation. 0–15 min : `State`; 15–35 min : `TimedState`; 35–50 min : construction et accès; 50–70 min : référence de base; 70–90 min : slicing; 90–110 min : `Trajectory`; 110–120 min : tests et bilan.

Compilation pendant tout le TD.

```
g++ -std=c++20 -Wall -Wextra -pedantic \
    -fsanitize=address,undefined -g \
    state.cpp timed_state.cpp trajectory.cpp main.cpp -o states
```

Au début, compiler seulement les fichiers déjà créés. Les sanitizers n'apportent une information que sur les cas effectivement exécutés.

1. State, une classe-valeur — 15 min

À FAIRE. Examiner puis compléter si nécessaire `State` : constructeur, `size()` et les deux versions de `operator[]`. Le stockage reste :

```
std::vector<double> values_;
```

TESTER. Vérifier une copie indépendante :

```
State a{3};
a[0] = 4.0;
State b{a};
b[0] = 9.0;
assert(a[0] == 4.0);
assert(b[0] == 9.0);
```

COMPRENDRE. Pourquoi aucun destructeur, constructeur de copie ou déplacement n'est-il écrit ? Quelle notion de la séance 4 l'explique ?

JALON 1 — RÈGLE DE ZÉRO

✓ copies indépendantes; ✓ aucun `new/delete`; ✓ aucune opération spéciale personnalisée.

2. TimedState est un State enrichi — 20 min

À FAIRE. Créer `timed_state.hpp` et `timed_state.cpp` :

```
class TimedState : public State {
public:
    TimedState(std::size_t dimension, double time);
    double time() const noexcept;
    double first() const;
private:
    double time_{};
};
```

Le constructeur initialise d'abord `State{dimension}`, puis `time_`. `first()` doit passer par `operator[]`.

TESTER. Construire et modifier :

```
TimedState u{3, 0.25};
u[0] = 1.0; u[1] = 2.0; u[2] = 3.0;
assert(u.size() == 3);
assert(u.time() == 0.25);
assert(u.first() == 1.0);
```

COMPRENDRE. Identifier la partie `State` et l'information ajoutée. Pourquoi `TimedState` ne peut-il pas lire directement `values_` ?

JALON 2 — OBJET DÉRIVÉ CONSTRUIT

✓ base construite avec la dimension; ✓ instant conservé; ✓ données de la base toujours privées.

3. Construction et accès à la base — 15 min

À FAIRE. Avant d'exécuter, prédire l'ordre pour un objet local : construction de `State`, construction de `TimedState`, puis destruction dans l'ordre inverse. Ajouter temporairement une trace dans les constructeurs et destructeurs pour vérifier, puis la retirer.

TESTER. Décommenter volontairement dans `first()` :

```
return values_[0];
```

Lire le diagnostic, puis restaurer `return (*this)[0];`.

COMPRENDRE. La restriction oblige le dérivé à utiliser l'interface de la base. La représentation de `State` peut ainsi changer sans modifier `TimedState`.

JALON 3 — BASE ENCAPSULÉE

✓ ordre prédit puis observé ; ✓ accès direct refusé ; ✓ accès public utilisé.

4. Réutiliser un algorithme existant — 20 min

À FAIRE. Utiliser les algorithmes déjà déclarés :

```
double l2_norm(const State& state);  
void scale(State& state, double alpha) noexcept;
```

TESTER. Exécuter :

```
TimedState u{3, 1.0};  
u[0] = 3.0; u[1] = 4.0;  
assert(l2_norm(u) == 5.0);  
  
const State& view{u};  
assert(l2_norm(view) == 5.0);  
scale(u, 2.0);  
assert(l2_norm(u) == 10.0);  
assert(u.time() == 1.0);
```

COMPRENDRE. La référence de base ne crée aucun nouvel objet. L'algorithme observe ou modifie la partie `State` du même `TimedState` et ignore simplement `time_`.

JALON 4 — SUBSTITUTION SANS COPIE

✓ algorithmes réutilisés ; ✓ même objet complet ; ✓ temps inchangé.

5. Observer le slicing — 20 min

À FAIRE. Construire volontairement une valeur de base et écrire une petite fonction par valeur :

```
void inspect(State state)
{
    state[0] = -1.0;
}

State snapshot{u};
inspect(u);
```

TESTER. Vérifier que `snapshot.size() == u.size()`, que modifier `snapshot` ne modifie pas `u`, et que `inspect` travaille sur une copie. Constater qu'un `State` ne fournit pas `time()`.

COMPRENDRE. Expliquer en une phrase : construire un `State` par valeur à partir d'un `TimedState` ne conserve que la partie `State`. Ce phénomène légal est appelé slicing.

JALON 5 — RÉFÉRENCE OU VALEUR

Référence de base = même objet ; valeur de base = nouvel objet de base.

6. Trajectory : composition — 20 min

À FAIRE. Créer une classe qui possède :

```
class Trajectory {
public:
    void append(TimedState state);
    const TimedState& at(std::size_t i) const;
    std::size_t size() const noexcept;
private:
    std::vector<TimedState> states_;
};
```

`append` refuse par `std::invalid_argument` tout état dont le temps n'est pas strictement supérieur au dernier. `at` peut déléguer à `vector::at`.

TESTER. Ajouter les temps 0.0, 0.5 et 1.0, puis vérifier taille et valeurs. Tester séparément l'ajout de 0.5 après 1.0 : une exception est attendue.

COMPRENDRE. Pourquoi `Trajectory` possède-t-elle un `vector` au lieu d'en hériter ? Elle n'est pas un `vector` ; son interface contrôle l'ordre temporel et peut changer de représentation.

JALON 6 — COMPOSITION ET INVARIANT

✓ temps croissants ; ✓ insertion invalide refusée ; ✓ aucune gestion manuelle de mémoire.

Bilan — 10 min

Répondre en quelques lignes :

1. différence entre possède-un et est-un ;
2. pourquoi `const State&` ne copie pas `TimedState` ;
3. ce que perd `State snapshot{u}` ;
4. pourquoi `State` garde `values_` privé ;
5. pourquoi `Trajectory` utilise la composition.

À rendre. Les sources, les tests exécutés avec les sanitizers et les cinq réponses courtes.